AEGIS: Data Analysis and Database Design

Technical Deliverable

Ilias Akritidis*, Georgios Mallousis*, Vasileios Sevetlidis

International Hellenic University, Polytechnic School
Department of Computer, Informatics and Telecommunications Engineering

iliakrit@ihu.gr geomallo@ihu.gr vasiseve@ihu.gr

1 Purpose and analytical scope

The analysis supports an educational question: how can a non-specialist connect a heritage site's preservation challenges with the monitoring instruments capable of detecting relevant changes? The resulting dataset represents threats, monitoring networks and interpretable events at Delos, Baltanás, Monti Lucretili, Sant'Antonio di Ranverso and Schenkenberg. Missions translate these records into bounded sensor-selection tasks. Recorded alarms provide examples against which proposed networks can be examined.

Two data domains support this process. The heritage domain consists of ARGUS records retrieved through its API and retained as JSON snapshots. The participation domain consists of gameplay and quiz records stored on the device, on a classroom server, or in an optional Supabase PostgreSQL database. The preparation pipeline reads the heritage domain; participation statistics are computed separately in the browser. The SQL schema examined here defines the application's participation store. The ARGUS data model is examined through its API representations.[1]

The scope includes record integration, quality screening, alarm selection, mission construction, descriptive participation measures and static database analysis. The analysis uses deterministic application rules. Database findings derive from static examination of the supplied DDL and consuming code. Establishing the configuration, contents and performance of a running Supabase instance requires deployment inspection.

2 Methodological basis

CRISP-DM describes an iterative process spanning business understanding, data understanding, preparation, modelling, evaluation and deployment [1]. These phases organise the AEGIS workflow (Table 1). Here, modelling consists of explicit transformation and selection rules.

Rahm and Do distinguish schema-level integration issues from instance-level problems such as missing values, invalid values and duplicate records [2]. This distinction informs the separation between relational reconstruction, alarm screening and retained quality flags. Filtering establishes suitability for the educational application. Assessing a record's operational validity requires the semantics and requirements of its original monitoring context.

Reproducibility also requires identifying the inputs and procedures associated with a result. Sandve et al. emphasise retaining software, data and intermediate information sufficient to reconstruct computational findings [3]. AEGIS retains snapshots, executable transformations, educational mappings and generated analysis notes. Reproduction therefore depends on both the ARGUS records and the version of the application content.

*Ilias Akritidis and Georgios Mallousis contributed equally to this work.

[1] Implementation sources: `tools/build_game_data.py`, `web/src/game/logic.ts`, `web/src/community/store.ts`, `web/src/community/index.ts`, `tools/class_server.py` and `supabase/schema.sql`. The analysis concerns the local `readyToSubmit` branch, with application baseline `b61c190`. Paths in implementation footnotes are relative to the repository.

Table 1: Organisation of the implemented analysis using CRISP-DM phases.

Phase	Application in AEGIS
Understanding the purpose	Relate preservation threats, sensor choices and interpretable alarms for non-specialist users.
Understanding the data	Inspect resources, metadata, identifiers, priorities, instrument positions and event types.
Preparation	Reconstruct joins, handle duplicate alarms and apply completeness, mapping and plausibility rules.
Modelling	Map threats and alarm rules to sensor cards, group alarm episodes and construct missions.
Evaluation	Validate references and content; test deterministic output, selection rules, scoring and record handling.
Deployment	Publish the gameplay pack; compute participation summaries through the configured record store.

3 Input data and logical relationships

3.1 Retained sources and temporal coverage

The principal inputs come from `snapshot_2026-10-12/`. Ten resources were retrieved between 15:00:44 and 15:02:45 on 12 October 2026, at UTC+03:00. Decision resources are supplied by the retained snapshot of 11 October. Table 2 reports counts recalculated from the files. Each listed file declares `complete=true`, and its recorded count agrees with the number of retained records.[2]

Completeness describes the retained API retrieval according to its metadata. Historical coverage and the extent of site observations require additional information about the upstream service. The resource timestamps document sequential acquisition. The AD-RT alarm timestamps span 28 August–8 October 2026. The generated notes contain a fixed sentence describing events as an incomplete sample, in conflict with the retained file's completeness flag. Counts reported here use the retained records and their actual metadata.

Table 2: Resources used in the retained analysis and their principal roles.

Resource	Records	Analytical role
`assets`	5	Heritage-site identity and pilot association.
`asset-states`	47	Associate installed devices with heritage assets.
`threats`	156	Identify ABC risk scenarios and priorities.
`threat-contexts`	1,350	Recover scenario evaluation histories.
`events`	1,543	Identify AD-RT alarms and their metadata.
`policies`	181	Retrieve policy context and threshold overrides.
`installed-instruments`	123	Link registered instruments to installed devices.
`installed-devices`	117	Supply asset-state links and position metadata.
`instruments`	117	Resolve instrument identity and type.
`instrument-types`	40	Map type names to educational sensor cards.
`decisions`	219	Supply recorded recommendations.
`decision-contexts`	217	Associate recommendations with alarm context.

The analysis operates on scenarios and derived monitoring events. The short retained readings series and the undocumented `sensor_coverage` field lie outside the selected inputs. Alarm reconstruction from original sensor time series would require a dedicated analysis.

3.2 Reconstruction of the heritage data model

API metadata may be a JSON-encoded string, an object or null. Parsing exposes the scenario, pilot, alarm and evaluation attributes. Unparseable metadata is retained internally as an unparsed value. Preparation and validation check the required relationships and application references after parsing.

[2] Core inputs: `snapshot_2026-10-12/*.json`. Supplemental decisions: `snapshot/decisions.json` and `snapshot/decision-contexts.json`. The fallback is implemented by `load_snapshot`; the game derives its displayed core snapshot date from the ten principal resources.

An installed-instrument record links to an installed device and an instrument. The device links to an asset state, which identifies the heritage asset. The instrument links to its type. This chain assigns each instrument to a site and then to a sensor card. Pilot identifiers in scenario metadata are matched to asset codes, with exactly one matching asset required. Risk histories are grouped by `scenario_key`. Recommendations are associated with alarms through pilot, original rule and timestamp. These logical relationships are reconstructed from the API representations. Verification of physical foreign-key constraints would require access to the ARGUS database definition.

3.3 Educational mappings and provenance

Sensor-card definitions specify labels, costs and instrument-type associations. Threat mappings identify primary and supporting cards, first by scenario and otherwise by risk category. Rule mappings specify sensor combinations, feature-name patterns and any rule-specific plausibility bounds. Monument content supplies explanations, curated missions and preferred replay events.[3]

The generated dataset distinguishes ARGUS-derived fields from authored content through provenance descriptions and `authored` or `needs_review` flags. Prices, sensor mappings, narrative explanations and illustrative deterioration rates originate in the application's educational configuration. Figure 1 presents the separation of inputs, transformations and outputs.

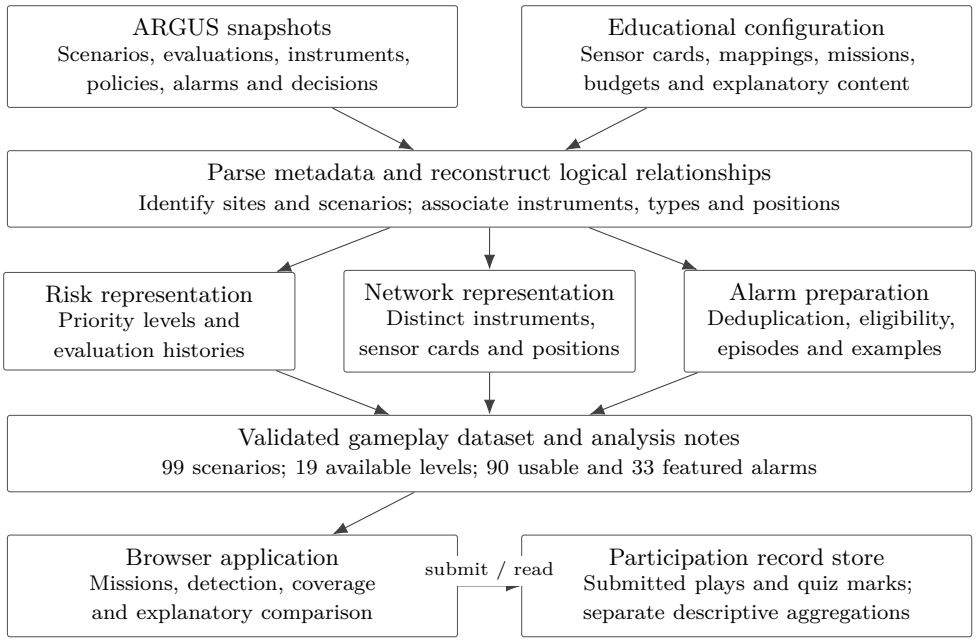


Figure 1: Data preparation and consumption. Participation records support browser-side descriptive analysis through the configured community store.

4 Data preparation and selection

4.1 Risk scenarios and evaluation histories

Records of kind `abc_risk_scenario` supply the risk scenarios. The inputs contain 99 scenarios. Priority labels map directly to the five levels: 30 low, 20 medium, 16 high, 23 extreme and 10 catastrophic. The dataset preserves the source ABC components and magnitude.

The builder recognises 1,133 records of kind `ira_threat_evaluation`, associated with 51 scenario keys. Evaluations are ordered by source timestamps. Consecutive evaluations above `inactive` are summarised as state intervals, with the largest probability within each interval retained. One scenario

[3]Content definitions: `content/sensor_cards.json`, `content/threat_sensor_map.json`, `content/rule_sensor_map.json`, `content/coins.json` and `content/monuments/*.json`.

has a raised history in these inputs: wind erosion at the House of the Masks on Delos, at the `watch` state. These results describe the recorded evaluation states within the retained period. Site-wide preservation risk requires assessment across the relevant evidence and time horizon.

4.2 Instrument identity and position

The 123 installation records resolve to 117 distinct instruments. Grouping by instrument identifier avoids counting repeated registrations as separate instruments. At Ranverso, 26 records describe 20 distinct instruments. Instruments are subsequently grouped by educational card for comparison with a proposed network. Coordinate pairs group the positioned instruments into 74 site-specific map points. Temporary positions, positions inherited from a sibling sensor and virtual instruments carry corresponding flags. Map points group instruments sharing coordinates.

4.3 Alarm identity and duplicate handling

The event file contains 337 AD-RT alarm records, alongside 1,134 IRA evaluation events and 72 EOP completion events. Alarm records comprise 217 expert-rule events (`EK`) and 120 model-derived events (`DL`).

Within each site, alarm identity is reconstructed using type, original rule, timestamp and the ordered variable-name sequence. Where no variable list is supplied, sorted feature-contribution names are used. Records are processed by increasing event identifier; the first supplies the retained values and subsequent matches contribute their identifiers. Differences in variable values set a quality flag. The resulting 257 unique alarms reflect grouping by this explicit identity key, including groups containing value variants.[4]

4.4 Interpretability and completeness

Expert-rule alarms retain their rule and variable. For a DL alarm, the largest supplied feature contribution identifies the principal signal. The alarm proceeds only if this contribution is at least 0.30 and its feature name matches a configured sensor-rule mapping. The 0.30 contribution threshold is an application screening parameter whose explanatory suitability requires evaluation against the upstream model.

For DL alarms, the analysed value is the model's anomaly score and the comparison limit is its anomaly threshold. The feature name supplies the sensor association, and the value retains its anomaly-score semantics. Subsequent processing requires a mapped rule, exactly one represented variable, numeric value and threshold, and a non-zero threshold. The inputs lose 66 unique alarms at the DL interpretation stage and 24 at the rule/completeness stage.

4.5 Plausibility and retained quality information

Physical range screening applies to EK measurements: humidity and soil moisture from 0 to 100, rain from 0 to 1,000, and temperature from -50 to 60. Rule-specific bounds further constrain derived quantities, including daily temperature range and one-hour temperature change. Exact zero humidity or soil moisture is treated as a suspect dropout. These eligibility rules depend on assumed variable semantics and units. Sensor-fault diagnosis requires calibration against the relevant instruments and operating conditions. DL anomaly scores undergo the applicable rule-specific checks.

This stage excludes 77 further alarms: 31 for humidity range, 19 for temperature range, 18 for the daily temperature-range rule, six for the one-hour change rule, and three for zero soil moisture. The resulting set contains 90 usable alarms. Disagreement with a current policy threshold and conflicting duplicate values are retained as flags alongside the selected values. The 37 AD-RT expert-rule policies provide current threshold context. Establishing the policy in force at a historical event requires the corresponding policy history.

[4] Alarm preparation: `build_alarms`, `dl_rule`, `policy_thresholds` and `link_instruments` in `tools/build_game_data.py`.

4.6 Episode construction and featured examples

Usable alarms are grouped by mapped rule and ordered by timestamp. A gap strictly greater than 48 hours starts a new episode. The displayed threshold multiple is

$$r(v,\tau) = \begin{cases} v/\tau, & v \geq \tau, \\ \tau/v, & v < \tau,\ v \neq 0. \end{cases} \tag{1}$$

The ratio is undefined for a zero value in the second case. Its analytical role is to order examples within the selection procedure. Interpretation in terms of physical damage, cross-sensor risk or prediction confidence would require a separately calibrated relationship.

An episode with at least three observations is flagged as approximately flat when

$$\bar{v} \neq 0 \quad \text{and} \quad \frac{\max(v) - \min(v)}{|\bar{v}|} < 0.02. \tag{2}$$

The first, peak and last alarm of each episode are selected, with roles combined when they identify the same alarm. Additional high-ratio examples bring the selection towards five alarms where sufficient records exist. A ten-alarm cap per site prioritises peaks before other roles. Flat-signal flags remain in the data. Level-replay selection filters out examples carrying a flat-signal flag. These operations produce a purposive selection of 33 illustrative alarms.

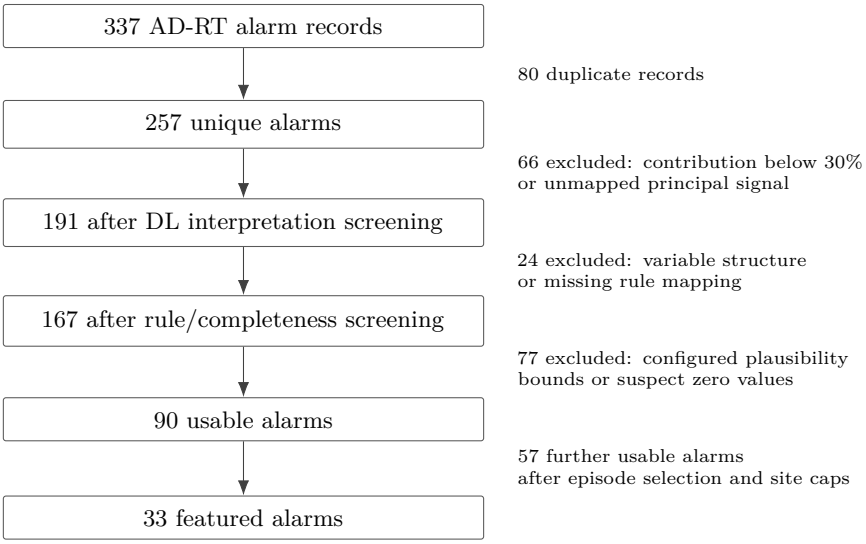


Figure 2: Sequential alarm eligibility and presentation selection. Exclusions apply to the preceding retained set. A further 57 usable alarms remain outside the featured selection.

5 Results and construction of learning activities

5.1 Distribution across heritage sites

Table 3 reports the recalculated site-level results. Overall, 90 of 257 unique alarms pass the eligibility rules (35.0%), and 33 of those 90 are featured (36.7%). These proportions describe selection on the retained dataset. Estimating detector accuracy, event prevalence or instrument reliability requires reference outcomes and an appropriate sampling design.

At Baltanás, 26 unique DL alarms fail the principal-contribution criterion and six have an unmapped principal signal. These screening outcomes account for the site's empty replay selection. The retained records document the occurrence of alarms. The site remains playable through preservation scenarios and network-design activities.

Table 3: Observed pipeline results. Exclusions are counted after deduplication.

Site	Scenarios	Raw	Unique	Excluded	Usable	Featured
Delos	36	46	37	6	31	8
Baltanás	7	41	32	32	0	0
Monti Lucretili	19	135	97	81	16	10
Ranverso	26	101	81	47	34	7
Schenkenberg	11	14	10	1	9	8
Total	99	337	257	167	90	33

5.2 Mission generation and scoring

The content provides 19 available levels: five at Delos, three each at Baltanás and Monti Lucretili, and four each at Ranverso and Schenkenberg. A mission contains three scenarios, using curated selections when present. Automatic selection otherwise traverses the requested priority followed by lower and higher priorities, using the builder's magnitude ordering within each group and avoiding repeated descriptions and scenarios without mapped cards. A curated replay specification is used where available; otherwise an eligible peak is preferred.

Detection is represented as a conjunction of sensor groups: the network must include at least one card from every required group. Coverage is full when a primary card is present, partial when only a supporting card is present, and absent otherwise. The score assigns 20 points to full coverage and eight to partial coverage, 25 to detection where a replayable alarm exists, and up to 15 to expenditure on useful cards. The total is normalised to the applicable components:

$$S = \operatorname{round}\left(100\,\frac{20F + 8P + A + E}{20N + 25I + 15}\right), \tag{3}$$

where N is the number of scorable threats, F and P are full and partial coverage counts, I indicates whether an alarm component applies, A is either 0 or 25 when applicable and zero otherwise, and E is the rounded efficiency component. At sites with an empty replay selection, the score is calculated from threat coverage and efficiency.[5]

Comparison with ARGUS's installed network provides descriptive feedback through the network-comparison view. Multi-day gameplay reuses recorded alarms for up to seven days, beginning with the level's event. Illustrative deterioration effects support educational interpretation. Conservation forecasting requires a validated predictive model. The separate recent-data feed reuses the published alarm filter; acquisition and freshness are specified in the API workflow deliverable.[6]

5.3 Published data contract

The generated `web/public/data/game_data.json` uses schema version 3. It contains snapshot metadata, provenance, card and rule definitions, alarm-screening parameters, quiz and observation content, and per-site scenarios, levels, instrument summaries, positions and featured alarms. Explanatory material and spatial references accompany these fields. The retained file occupies 384,489 bytes; the builder's default JSON serialisation is approximately 304.8 KiB. This size excludes the remaining application, imagery, audio and three-dimensional assets.

The accompanying `docs/data_notes.md` records counts, exclusions, examples and assumptions. Both outputs agree with fresh execution against the retained inputs. Automatic selection of the newest snapshot means that reproducing these results requires explicitly selecting the dated input directory, together with the original supplemental snapshot and the same content files.

[5] Scoring, sensor-group detection, network comparison and multi-day selection: `web/src/game/logic.ts`. Score labels use thresholds 85, 60 and 30, which define the application's educational feedback categories. Establishing their relationship to learning outcomes requires empirical evaluation.

[6] Companion deliverable: `docs/submission/api-workflow/main.tex`. Filter contract: `alarm_filter` in `web/public/data/game_data.json`.

6 Participation database

6.1 Schema and analytical units

The shared store is defined by `supabase/schema.sql`. It creates `public.plays` and `public.quizzes`, each with a generated `bigint` identity primary key. A play row records one submitted gameplay result; a quiz row records one submitted post-game quiz with an optional pre-game response array. The two tables are defined independently. Foreign-key relationships to users, missions and ARGUS resources remain undefined in the supplied schema.

The shared `monument`, `class_code` and `day` attributes provide grouping dimensions. Session-level association between a play and a quiz requires an additional shared identifier. Joining on the grouping dimensions can produce multiple row combinations. Pre/post responses are paired within each quiz row, supporting comparison at the quiz-submission level. Figure 3 shows the implemented schema boundary.

PostgreSQL supports relational constraints together with `jsonb` documents [4]. AEGIS keeps stable analytical dimensions in scalar columns and represents sensor selections and quiz marks as variable-length structures. The supplied constraints define only part of the intended record semantics (Tables 4 and 5).

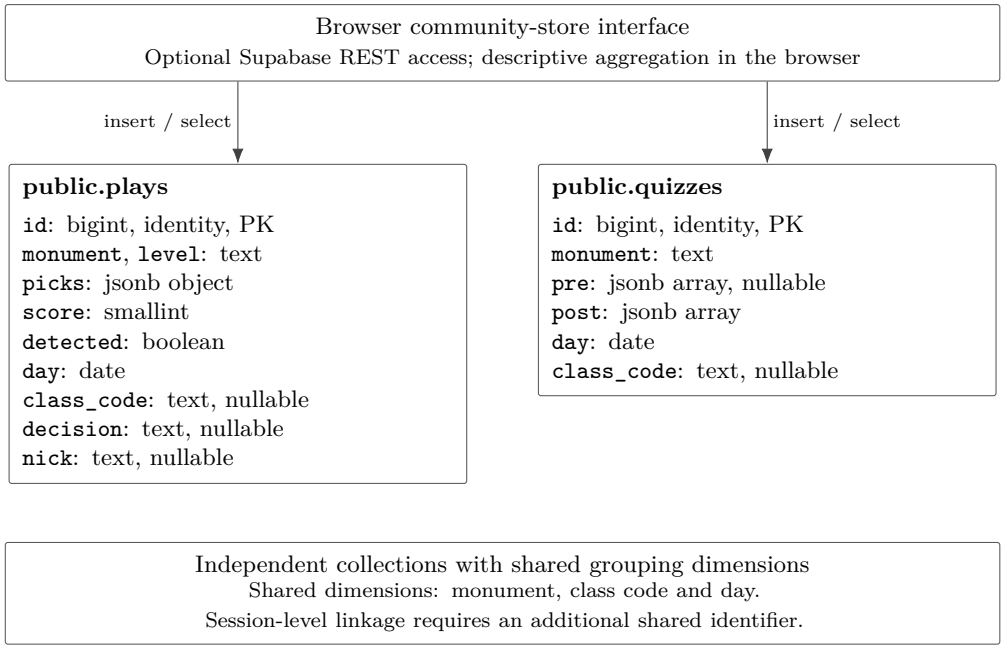


Figure 3: Participation database and consumers. Each table has an independent primary key. Monument, class code and day support grouping within each collection.

Table 4: Fields and constraints of `public.plays`.

Field	SQL type	Meaning and declared constraint
`id`	`bigint`	Generated identity; primary key.
`monument`	`text`	Required; 3–20 lowercase ASCII letters. Validation checks the identifier's syntax.
`level`	`text`	Required; one of low, medium, high, extreme or catastrophic.
`picks`	`jsonb`	Required object; stored size below 2,000 bytes. Entry-level semantics depend on client validation.
`score`	`smallint`	Required; client-calculated integer from 0 to 100.
`detected`	`boolean`	Required; client-reported detection outcome.
`day`	`date`	Required; 1 January 2026 through the database's current date plus one day when checked.
`class_code`	`text`	Optional; six uppercase characters from the configured unambiguous alphabet.
`decision`	`text`	Optional; 2–30 lowercase letters or underscores. Validation checks the identifier's syntax.

Table 5: Fields and constraints of `public.quizzes`.

Field	SQL type	Meaning and declared constraint
`id`	`bigint`	Generated identity; primary key.
`monument`	`text`	Required; same syntactic site rule as `plays`.
`pre`	`jsonb`	Optional array of at most 20 elements.
`post`	`jsonb`	Required array of at most 20 elements.
`day`	`date`	Required; same date window as `plays`.
`class_code`	`text`	Optional; same six-character pattern as `plays`.

6.2 Integrity, indexes and schema evolution

PostgreSQL provides primary keys, foreign keys, uniqueness, non-null requirements and check constraints as distinct integrity mechanisms [5]. The DDL combines these mechanisms selectively. Identity keys distinguish stored rows. Repeated submissions of the same logical result receive independent row identities. The declared checks permit arbitrary keys and values within the bounded `picks` object. Quiz arrays may be empty, contain non-boolean elements or have unequal lengths. Enforcing the intended card and response semantics for direct submissions requires additional database constraints.

Three explicit indexes support common filters: `plays(monument, level)`, `plays(class_code)` and `quizzes(class_code)`. Primary keys additionally provide identifier indexes. JSON-content indexing and materialised aggregation remain available as possible schema extensions. The REST reader requests selected fields ordered by descending identifier, with a limit of 2,000 rows. Summaries concern the returned subset, whose size also depends on service configuration. Database-wide analysis requires complete retrieval or server-side aggregation. Query-plan inspection and performance measurement lie outside this assessment.

Compatibility statements add `decision` and `nick` columns to older projects and replace the nickname check. The browser retries an affected play operation without `nick` following HTTP 400, accommodating a deployment lacking that column. The HTTP status code triggers this compatibility behaviour. Confirmation of the deployed schema requires direct inspection.[7]

6.3 Access model and identity semantics

Row-level security is enabled on both tables. The supplied policies allow the `anon` role to select all rows and insert rows satisfying table constraints. The script defines policies for selection and insertion; update and delete operations remain subject to PostgreSQL's default-deny behaviour for the anonymous role under this policy set. PostgreSQL evaluates row policies in addition to ordinary privileges; roles with bypass rights and normally table owners are outside that restriction [6]. Explicit grants, additional existing policies and project-level settings require deployment inspection. This analysis describes the access configuration expressed in the supplied DDL.

The class code is an equality filter in application requests. The select policy permits access to rows across classes. Establishing student or teacher identity requires an authenticated submission mechanism. Scores retain the status of client-reported gameplay results.

The nickname format permits a user-chosen name and a six-character suffix derived in the browser from that name and a password. The submitted nickname contains the chosen name and derived suffix; the password remains in the browser-side computation. Legacy records may use an animal-number label. These formats support leaderboard grouping. Authentication and anonymity require assessment of the complete identity and data-handling design. A chosen name can itself contain identifying information. The schema's introductory comment labels the records anonymous. Privacy assessment should account for the permitted user-chosen names.[8]

[7] DDL and indexes: `supabase/schema.sql`. REST column selection, descending order, requested limit and nickname compatibility: `SupabaseCommunityStore` in `web/src/community/store.ts`.

[8] Nickname construction and normalisation: `web/src/community/nickname.ts`; optional account flow: `web/src/community/account.ts`. Password-storage security and provider logging require separate deployment assessment.

6.4 Device, classroom and shared storage

The default device store retains up to 500 records of each local collection and uses in-memory fallback when browser storage is unavailable. The classroom server writes JSON Lines files, retains up to 50,000 records per file and returns at most the newest 5,000 matching records. It defaults to loopback access, with LAN operation enabled explicitly.

Classroom validation applies the following entry-level checks: sensor selections are limited to 40 entries with integer counts from 1 to 99, and quiz marks must be non-empty boolean arrays of at most 20 elements. Catalogue membership requires additional validation beyond the field-name patterns. Each storage mode applies its documented acceptance rules to the shared record contract.

When shared storage is configured, a local copy and pending queue support retry and fallback. Reads can switch to local records after a shared-store failure. Each successful insertion creates a new row, and an uncertain network outcome followed by retry can produce repeated records. Idempotent submission handling would require a stable submission key and an associated uniqueness constraint. Statistics describe the active storage scope and count submitted records.

7 Participation measures and interpretation

For n returned play records, the browser computes mean score, the fraction reporting successful detection, and the proportion of networks containing each sensor card:

$$\bar{S} = \frac{1}{n}\sum_{i=1}^{n} S_i, \qquad p_{\text{det}} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}(d_i), \qquad p_c = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}(q_{ic} > 0). \tag{4}$$

Here q_{ic} is the selected quantity of card c. The last measure expresses the frequency with which networks include a card. Decision counts include only records with a decision. Empty collections return zero for means and shares. The accompanying record count identifies empty collections.

For quiz summaries, the paired set contains rows with a pre-game array. Mean pre- and post-game correct-answer counts use that same set. Per-question comparison additionally requires both arrays to match the requested question count. The general summary accepts the recorded array lengths. The per-question analysis uses its specified question count to establish a common response structure.[9]

The stored quiz marks support descriptive comparison within each submission. Analysis of quiz version, individual answer choices, session identity, intervention exposure and verified participants would require additional fields. Repeat play, selective completion, differing missions, changes in quiz content and bounded retrieval can affect comparisons. Estimating causal learning effects and their persistence requires an appropriate evaluation design and follow-up observations. The user-experience questionnaire cited in the abstract forms a separate dataset. The available participation evidence comprises the schema and consuming code; analysis of player sample sizes and outcomes requires populated records.

8 Reproduction and verification

8.1 Reconstructing the retained analysis

The core builder requires Python 3.9 or later and uses the standard library. These commands select the retained snapshot explicitly; the first validates without writing and the second regenerates output:

```
python3 -I tools/build_game_data.py --snapshot snapshot_2026-10-12 --check
python3 -I tools/build_game_data.py --snapshot snapshot_2026-10-12
```

The original snapshot remains necessary for supplemental decisions, and the corresponding content, translation and spatial files must be retained. Acquisition of a new snapshot is unnecessary for reproducing these results. Newer snapshots or changed mappings can legitimately produce different output.

[9] Aggregations: `summarize`, `decisionCounts`, `summarizeQuiz` and `quizByQuestion` in `web/src/community/store.ts`.

Read-only SQL inspection queries appear in Appendix A. They provide a procedure for comparing a deployed database with the supplied schema. Execution against a remote service remains a deployment verification step. Applying DDL or changing access policies lies outside this verification.

9 Limitations and conclusion

The heritage analysis depends on source semantics and units and on application mappings requiring expert review. Duplicate resolution retains the first matching record and flags value conflicts. Episode boundaries, screening ranges, feature-contribution eligibility and selection caps are explicit configuration choices, whose effects require sensitivity analysis. Selection favours interpretable examples. Estimation of environmental-event frequency and upstream detector performance requires a suitable sampling design and reference outcomes. The participation database provides a compact record of submitted choices and quiz marks. Its schema supports descriptive summaries of submitted records. Session linkage, authenticated participant counting, harmonised validation and duplicate prevention require additional schema and application mechanisms. These conditions define the scope of interpretation. Within this scope, AEGIS provides a traceable transformation from retained monitoring records to 19 playable levels, with source-linked scenarios, instrument comparisons and selected alarm examples. Separating heritage observations, educational configuration and participation records makes the analytical basis inspectable and supports subsequent expert review and evaluation.

References

[1] IBM, *IBM SPSS Modeler CRISP-DM Guide*, version 18.5.0, ch. 1. https://www.ibm.com/docs/en/SS3RA7_18.5.0/pdf/ModelerCRISPDM.pdf.

[2] E. Rahm and H. H. Do, *Data Cleaning: Problems and Current Approaches*, IEEE Data Engineering Bulletin, vol. 23, no. 4, pp. 3–13, 2000. https://pubs.dbs.uni-leipzig.de/dc/files/Rahm2000DataCleaningProblemsand.pdf.

[3] G. K. Sandve, A. Nekrutenko, J. Taylor and E. Hovig, *Ten Simple Rules for Reproducible Computational Research*, PLOS Computational Biology, vol. 9, no. 10, e1003285, 2013. https://doi.org/10.1371/journal.pcbi.1003285.

[4] PostgreSQL Global Development Group, *PostgreSQL 17 Documentation*, sec. 8.14, JSON Types. https://www.postgresql.org/docs/17/datatype-json.html.

[5] PostgreSQL Global Development Group, *PostgreSQL 17 Documentation*, sec. 5.5, Constraints. https://www.postgresql.org/docs/17/ddl-constraints.html.

[6] PostgreSQL Global Development Group, *PostgreSQL 17 Documentation*, sec. 5.9, Row Security Policies. https://www.postgresql.org/docs/17/ddl-rowsecurity.html.

A Read-only database inspection

These PostgreSQL queries provide inspection procedures for an authorised database connection. Inspection of policies should be accompanied by inspection of table grants and the effective API role.

```
-- Column types and nullability.
SELECT table_name, column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name IN ('plays', 'quizzes')
ORDER BY table_name, ordinal_position;

-- All policies, including any added outside the supplied DDL.
SELECT tablename, policyname, roles, cmd, qual, with_check
FROM pg_policies
WHERE schemaname = 'public'
  AND tablename IN ('plays', 'quizzes');

-- Index definitions.
SELECT tablename, indexname, indexdef
FROM pg_indexes
WHERE schemaname = 'public'
  AND tablename IN ('plays', 'quizzes');
```

A database-side summary operates on all rows visible to the executing role and describes submissions and client-reported outcomes:

```
SELECT monument, level,
       COUNT(*) AS submissions,
       AVG(score) AS mean_submitted_score,
       COUNT(*) FILTER (WHERE detected)::numeric
         / NULLIF(COUNT(*), 0) AS reported_detection_share
FROM public.plays
GROUP BY monument, level
ORDER BY monument, level;
```

The queries analyse each table independently. A session-level join would require a shared session identifier.